Hierarchical UAV Autonomous Navigation Algorithm based on Event-Triggered Deep Reinforcement Learning

Weisheng Chen, Yuntao Xue

***Abstract*—In unknown environments, achieving autonomous navigation for unmanned aerial vehicles (UAVs) is a complex task. Ensuring that UAVs reach their destinations safely and efficiently remains a critical challenge. When obstacles are densely distributed, conventional motion planning algorithms often face difficulties in finding viable solutions due to increased problem complexity. Deep reinforcement learning (DRL)-based navigation strategies are known for their generalization capabilities and robustness. However, the trajectories generated through end-to-end reinforcement learning (RL) approaches tend to lack smoothness and may not align with the UAVs' dynamic feasibility requirements. This study proposes a hierarchical navigation framework by combining event-triggered deep reinforcement learning with traditional motion planning techniques. In the proposed framework, the DRL module receives raw sensor input to generate local goals, while the lower-level classical planner ensures that the paths leading to these goals are smooth and collision-free. The event-triggered Recurrent Soft Value Gradient (RSVG) algorithm activates goal generation only when obstacles are detected, optimizing computational efficiency and improving real-time decision-making. Extensive experiments are conducted in simulated environments with randomly placed static obstacles to validate the effectiveness of the proposed framework.**

***Index Terms*—hierarchical UAV Navigation, deep reinforcement learning, motion planning, event-triggered ,partially observable Markov decision process**

I. Introduction

OVER the past few decades, autonomous unmanned aerial vehicles (UAVs) have become indispensable due to their agility, lightweight structure, and multifunctional roles, particularly in applications such as emergency response and logistics [1]–[4]. Intelligent UAVs are increasingly used to replace humans in dangerous tasks, making autonomous navigation a fundamental requirement for performing diverse missions. Traditional autonomous navigation systems consist of modules like localization, global path planning, and local path optimization, which rely on sensor data to predict states and make navigational decisions. However, these methods often require predefined environmental settings, and building detailed maps can demand considerable computational resources. Therefore, the development of mapless motion planning algorithms with enhanced adaptability to complex environments has become crucial. Unlike classical motion planning methods , which rely on prior knowledge of the environment, Deep Reinforcement Learning (DRL) allows UAVs to learn navigation strategies through direct interaction with the environment, enabling real-time decision-making in unknown and dynamic scenarios. DRL can inherently handle obstacle avoidance, goal prioritization, and kinematic constraints without the need for manually designed rules.

The advancement of DRL has enabled UAVs to map raw sensor data directly to control commands, facilitating navigation in unknown environments without relying on pre-built maps [5]. Through interactive training across diverse scenarios, DRL-based systems exhibit high generalization capabilities. Mnih et al. [6] first employed the Deep Q-Network (DQN) algorithm to manage discrete actions in video games, while Lillicrap et al. [7] extended these concepts to continuous control using the deep deterministic policy gradient (DDPG) framework. Heess et al. [8] introduced recurrent deterministic policies for more complex scenarios, and Pham et al. [9] designed reinforcement learning (RL) algorithms tailored for UAV navigation in dynamic environments. Nevertheless, end-to-end control strategies often generate trajectories that are less smooth and energy-efficient than those produced by traditional planning algorithms, leading to increased operational rigidity and energy consumption.

Within this context, Partially Observable Markov Decision Processes (POMDPs) serve as a foundational modeling tool for navigation under uncertainty. However, standard POMDP formulations assume a one-step action-to-transition mechanism, which is insufficient for describing multi-layer navigation systems where high-level decisions must be physically realized through low-level planners before any state transition occurs.

To bridge the gap between the decision-making capabilities of DRL and the trajectory smoothness offered by classical planning, recent studies have explored hybrid approaches that integrate DRL with motion planning. A representative example is the subgoal-driven hierarchical architecture proposed in [10], which decomposes navigation into two layers. The upper layer leverages attention-based DRL to predict subgoals, while the lower layer executes smooth motion commands toward these subgoals, effectively decoupling obstacle avoidance from

Manuscript received April xx, 20xx; revised August xx, 20xx. This work was supported by the National Natural Science Foundation of China under Grants 92271101 and 62073254. *(Corresponding author: Weisheng Chen.)*

Weisheng Chen is with the School of Information Mechanics and Sensing Engineering, Xidian University, Xi'an 710071, China (e-mail: wshchen@126.com).

Yuntao Xue is with the School of Aerospace Science and Technology, Xidian University, Xi'an 710071, China, and also with the College of Electronic Information Engineering, Taiyuan University of Technology, Taiyuan 030024, China (e-mail: ytxue@stu.xidian.edu.cn).

motion control. This framework enables robust, collision-free navigation in dynamic environments using only 2D LiDAR and global path information. By incorporating classical motion execution principles, the approach significantly reduces the jerkiness typically observed in pure DRL outputs. However, in such hierarchical architectures with fixed-interval triggering, a temporal coordination challenge arises between the DRL decision-making layer and the motion control layer. While the UAV follows its current subgoal trajectory, the DRL policy enters a Decision Silence Period (DSP)—a state in which no new decisions are made until the vehicle nears the subgoal or a replanning condition is triggered. This design introduces a critical risk: delayed response to environmental changes. During the DSP, newly emerging obstacles may go unnoticed by the DRL layer, and the lower-level motion planner, restricted by the fixed subgoal, may produce unsafe trajectories in response.

This limitation stems fundamentally from the use of time-triggered control (TTC) mechanisms in existing hybrid frameworks, where decisions are executed at predetermined intervals regardless of environmental dynamics. Conversely, event-triggered control (ETC) activates decision-making only when specific conditions are met, thereby conserving computational resources and enabling real-time responsiveness. Recent advances in ETC-RL integration have demonstrated success in applications ranging from robotic manipulation to autonomous driving, yet its potential in hierarchical UAV navigation remains underexplored.

To address these challenges, we propose ETRLNav, a novel hierarchical framework that introduces adaptive event triggering to coordinate DRL-based decision-making with classical motion planning. Unlike fixed-interval hybrid systems, ETRLNav employs a dual triggering mechanism where the DRL layer dynamically generates subgoals only when environmental changes (e.g., new obstacles or target displacements) exceed predefined event thresholds, thereby eliminating unnecessary computations during stable navigation phases, while the motion planning layer activates trajectory replanning upon detecting deviations from the current path or violations of velocity safety margins. This design achieves two interconnected innovations: First, temporal coordination is realized by synchronizing decision-making events with environmental dynamics, resolving the DSP dilemma through persistent situational awareness in the DRL layer even during motion execution. Second, layer coupling establishes a bidirectional feedback loop, allowing the motion planner's execution status to directly influence the DRL module's event thresholds, thereby enabling context-aware triggering that adapts to both environmental and system-level constraints.

The framework is grounded in an enhanced POMDP formulation that redefines hierarchical decision abstraction. Unlike conventional POMDPs using low-level control inputs (velocities/angles) for immediate state transitions, our model reconfigures the action space as the aforementioned subgoals. Through this dual innovation in framework architecture and decision modeling, ETRLNav achieves efficient coordination between cognitive-level strategic decisions and physical-level motion constraints. The framework's architecture is illustrated in Figure 1.

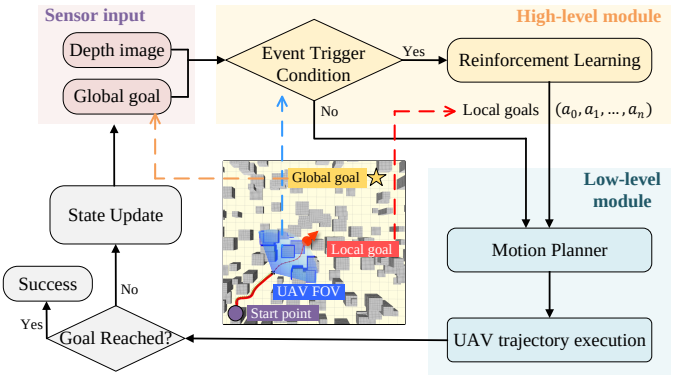


Fig. 1. The RLPlanNav framework consists of an upper-level DRL decision module and a lower-level motion planning module. The DRL module selects intermediate targets based on sensor readings, which the motion planner uses to generate dynamically feasible and smooth trajectories. This process repeats iteratively until the UAV reaches its final target.

The contributions of this paper are as follows:

1) We propose the hierarchical ETRLNav framework, which combines event-triggered DRL and classical motion planning techniques, taking advantage of both approaches to enhance UAV navigation in unknown environments.

2) An event-triggered fast Recurrent Soft Policy Gradient (FRSPG) algorithm is introduced to optimize the generation of sub-goals based on environmental conditions, improving decision-making efficiency and increasing the success rate of the navigation tasks.

3) An enhanced POMDP model is developed to facilitate UAV navigation in uncertain environments, leveraging the trained DRL module for effective problem-solving.

The remainder of this paper is structured as follows. Section II reviews related work. Problem representation and preparatory work are introduced in Section III. Section IV presents the hierarchical UAV navigation framework and the event-triggered deep reinforcement learning algorithm. Section V evaluates the performance of the algorithm in experiments. Section VI concludes the paper.

II. Related Work

A. Classical Motion Planning

Autonomous UAV navigation requires fundamental modules such as localization, mapping, path searching, and trajectory generation. Path searching and trajectory generation can be considered as the front-end and back-end of motion planning, respectively, with mapping serving as a prerequisite for these processes. The objective of path searching is to identify a safe route consisting of discrete waypoints from the start to the destination. Search-based methods, such as A* and Dijkstra's algorithm [11], and sampling-based methods, such as the Probabilistic Roadmap (PRM) [12] and Rapidly-Exploring Random Tree (RRT) [13], are commonly employed. However, trajectories derived from these algorithms lack temporal information. To ensure dynamic feasibility, these initial trajectories must be parameterized in time. Chen et al. [14] generated a safe corridor composed of 3D cubes within an octree-based

environment, followed by using quadratic programming (QP) to generate smooth trajectories within this flight corridor. In a separate study, Gao et al. [15] employed a 3D laser rangefinder to incrementally build point cloud maps and generate trajectories directly on these maps. Given the limited sensing range of UAVs, although rapid responses can be achieved through repetitive local planning or reactive obstacle avoidance, UAVs with restricted global information may become trapped in local minima. To mitigate this, some researchers adopt optimistic assumptions about the environment, treating unknown regions as free space, which may lead to hazardous collisions. Oleynikova et al. [16] combined the concept of local exploration with a local planner for UAV navigation in unknown forests, using an intermediate-point selection strategy akin to the "next-best-view" method, which selects future points that offer the greatest gain. In contrast, our work employs deep reinforcement learning to train the local target generation strategy, yielding higher success rates compared to optimistic search methods.

B. Event-triggered Deep Reinforcement Learning

There has been extensive research on classical event-triggered control, with early studies primarily focused on stability. With the rapid advancement of deep learning, learning-based control methods have emerged to implement optimal event-triggered control. For instance, a model-free reinforcement learning method was proposed for event-triggered control in continuous-time linear systems, achieving optimal performance. An actor-critic method was employed to learn the event-triggered controller and to guarantee system stability. However, in these studies, the communication triggering conditions are predefined and not learned dynamically. A model-based reinforcement learning approach was introduced to jointly learn the optimal event-triggered controller and the system model, with fixed communication thresholds. Additionally, another approach for learning event-triggered controllers from data leveraged deep reinforcement learning to learn both control and communication behaviors simultaneously. For autonomous driving path tracking, event-triggered model predictive control was developed, employing DRL to train the triggering conditions while using model predictive control for motion generation. Similarly, Lu et al. developed an event-triggered deep Q-network for autonomous driving, which does not require explicitly trained triggering conditions. These methods are all end-to-end learning approaches, and the discrete actions generated by such approaches often lack coherence, resulting in less smooth trajectories.

C. Hierarchical Reinforcement Learning Navigation

Recent studies have explored the integration of learning-based methods with traditional navigation strategies to improve robot navigation performance. Rana et al. [17] enhanced a suboptimal classical controller with a learned residual strategy, enabling the system to switch back to the classical controller when the learned policy underperforms, particularly in cluttered indoor navigation tasks. Another study by Faust et al. [18] combined reinforcement learning (RL)-based short-range local navigation strategies with Probabilistic Roadmaps (PRM) to enable long-range navigation. While these methods rely on RL to learn control strategies, the high dimensionality of the state space demands substantial computational resources. To capitalize on DRL's decision-making capabilities, Bansal et al. [19] proposed a learning-based module to generate waypoints, followed by a model-based planner to produce smooth, dynamically feasible trajectories, a framework they named LB-WayPtNav. This approach was experimentally tested on ground robots. Another related work, ReLMoGen [20], integrated motion generation into reinforcement learning for mobile manipulation, where the system learns to generate sub-goals from high-dimensional observations, followed by a motion generator that plans and executes precise movements to reach these sub-goals. The motion planner in these systems can be replaced flexibly, enabling adaptation to different environments. However, these approaches primarily target ground robots. The most relevant work to ours is the event-triggered hierarchical planner proposed by [21] for UAV navigation in unknown environments, in which a high-level deep neural network (DNN) handles reactive replanning, while a low-level DNN performs target tracking. Inspired by this framework, we propose a tightly coupled hierarchical UAV navigation framework that integrates reinforcement learning for local target generation with classical motion planning for trajectory computation. In parallel, transformer-based reinforcement learning methods and uncertainty-aware planning frameworks [22], [23] have emerged, leveraging attention mechanisms and confidence estimation to improve policy reliability in high-dimensional. While powerful, such methods often incur substantial computational overhead and are not always suitable for resource-constrained UAV platforms. Unlike these heavyweight architectures, our framework emphasizes decision efficiency through sparse event-triggered updates and compact RNN-based policies, enabling practical deployment without sacrificing performance.

Compared with the aforementioned methods, the proposed ETRLNav framework introduces several key advantages. Unlike fully end-to-end learning systems that often suffer from unstable behavior and lack of trajectory smoothness, ETRLNav separates the decision-making and motion execution layers by combining DRL-based sub-goal selection with classical trajectory planning. This design improves dynamic feasibility and ensures smoother and more natural flight. Moreover, while prior hierarchical methods such as LB-WayPtNav and ReLMoGen rely on fixed time intervals for decision-making, ETRLNav adopts an event-triggered control mechanism that adapts to environmental complexity, reducing unnecessary computation and improving real-time responsiveness. These innovations jointly address the limitations of both traditional planners and purely learning-based approaches in UAV navigation.

III. System Model and Problem Formulation

This section formulates the UAV navigation problem in unknown, complex environments. We first establish the UAV

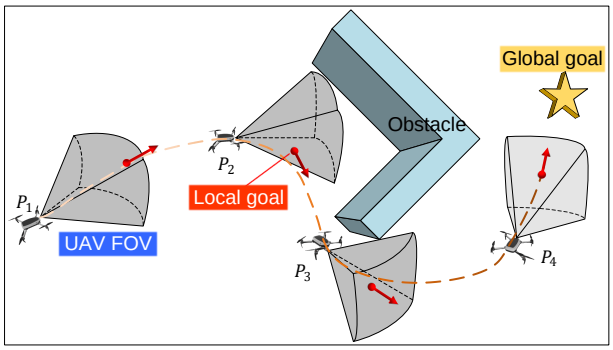


Fig. 2. Schematic illustration of a UAV with a limited field of view relying on local target guidance in an unknown environment.

dynamic model, followed by an outline of the navigation objectives and performance metrics.

A. UAV Model

In this study, the UAV momentum during flight is neglected, assuming that steering actions take effect immediately. We extend the previous 2D modeling approach to 3D, and the UAV dynamics are described as:

$$\begin{cases} \vartheta_{t+1} = \vartheta_t + \rho_t \\ \varphi_{t+1} = \varphi_t + \tau_t \\ x_{t+1} = x_t + v \times \cos(\vartheta_{t+1}) \cos(\varphi_{t+1}) \\ y_{t+1} = y_t + v \times \sin(\vartheta_{t+1}) \cos(\varphi_{t+1}) \\ z_{t+1} = z_t + v \times \sin(\varphi_{t+1}) \end{cases} \tag{1}$$

where ϑ_t and φ_t represent the UAV's directional angles in the horizontal and vertical planes, respectively, while ρ_t and τ_t correspond to the steering signals for these angles. The parameter v denotes the UAV's velocity.

B. Problem Description

This work addresses the problem of UAV navigation in unknown, complex environments, with the objective of ensuring safe, efficient movement. Here, safety refers to a collision-free navigation process, while efficiency implies that the navigation path is both as short as possible and dynamically feasible. A significant challenge arises from the lack of prior maps and the presence of dense obstacles, such as those found in forests or highly developed urban areas. For example, as shown in Figure 2, the UAV is directed towards its target, but at the densely obstructed location P_2, the UAV's limited observability prevents it from identifying a suitable direction and results in it becoming trapped. The UAV's partial observability is constrained by its camera or rangefinder, which have limited fields of view (FOV) in both horizontal and vertical planes—a realistic constraint. We assume that RGB-D cameras or rangefinders can accurately determine the UAV's state, and the environment is static.

The objective is to guide the UAV from any starting position $p_0 = (x_0, y_0, z_0)$ to a target location $p_g = (x_g, y_g, z_g)$, ensuring a collision-free, smooth trajectory through unknown, complex environments. At each time step t, the UAV receives environmental data and pose information, denoted as observation o_t. Our strategy is designed to use this observation information to generate local targets, which are then processed by the motion planner to generate trajectories that meet dynamic constraints. This process repeats iteratively, guiding the UAV towards its goal efficiently. As illustrated in Figure 2, when the UAV detects obstacles at P_2, the DRL-based strategy generates a local target $p_l = (x_l, y_l, z_l, \phi_l, \theta_l, \psi_l)$ (represented by red dots for location and red arrows for direction), enabling the UAV to escape the local trap. This mapless obstacle avoidance method leverages implicit environmental knowledge learned from past trajectories using recurrent neural networks.

Although the environment is assumed to be static in structure, the UAV operates under limited field-of-view (FOV) constraints, relying solely on onboard sensors such as RGB-D cameras or rangefinders. As a result, the UAV perceives only partial and gradually revealed segments of the environment. From the agent's perspective, this creates a form of dynamic interaction, where new obstacles may appear suddenly as the UAV moves, and past information may no longer be valid.

This setting effectively mimics the challenges of dynamic environments in terms of incomplete and time-varying observations. Our framework addresses this by leveraging memory-based recurrent policies and an event-triggered mechanism that adaptively generates sub-goals in response to newly perceived obstacles. Although designed for static layouts, this design inherently provides adaptability to quasi-dynamic and partially observable environments. Potential extensions to handle fully dynamic obstacles (e.g., moving agents) could include integrating real-time object tracking modules and predictive motion models into the observation and planning components.

The UAV's navigation performance is evaluated based on the following metrics: success rate (reaching the target within 0.5m without collisions), average time to reach the target (for successful cases), and average acceleration and jerk along the trajectory. The latter metric measures the smoothness and power efficiency of the UAV's operation. All components of this method must be sufficiently fast to enable real-time, online execution.

C. Event-Triggered Control

A nonlinear discrete-time system disturbed by additive Gaussian noise can be described as:

$$\begin{aligned} x_{k+1} &= f(x_k, u_k) + v_k \\ y_k &= x_k + w_k, \end{aligned} \tag{2}$$

where k represents the discrete time index, and v_k and w_k are independent Gaussian random variables, with their probability density functions given by $\mathcal{N}(v_k; 0, \Sigma_p)$ and $\mathcal{N}(w_k; 0, \Sigma_m)$, respectively.

In event-triggered control, actions are not computed and updated at every moment but are updated when specific conditions are met. The input is defined based on whether an update decision γ_k is made, depending on a triggering rule $\mathcal{C}_k$. Intuitively, if the current system state deviates significantly from the last controlled state, a control decision is triggered.

In this work, our goal is to use DRL methods to learn strategies for generating event-triggered sub-goals (i.e., control rules $\mathcal{K}_k$).

IV. Event-Triggered Hierarchical Navigation Framework: Integrating DRL and Motion Planners

This section will build an improved POMDP model, introduce deep reinforcement learning algorithms, and introduce motion planning algorithms.

A. ETRLNav Navigation Framework

This paper models the UAV navigation task as a time-discrete Partially Observable Markov Decision Process (POMDP), defined by the septuple $(S, A, T, R, \Omega, O, \gamma)$. Here, S represents the state space, A the action space, T denotes the state transition probabilities, $R : S \times A \to \mathbb{R}$ is the reward function, Ω is the observation space, O represents the observation transition probabilities, and $\gamma \in [0, 1)$ is the discount factor. At each time step t, the environment is in state $s_t \in S$, and the agent selects an action $a_t \in A$, leading to a transition to state s_{t+1} according to the probability distribution $T(s_{t+1}|s_t, a_t)$. The agent then receives an observation o_t from $O(o_t|s_{t+1}, a_t)$ and obtains a reward r_t. The goal of the agent is to choose actions that maximize the expected cumulative discounted reward $E\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$. Since the agent cannot directly observe the true state s_t, it must infer it from the observations Ω using the observation model $O(o_t|s_{t+1}, a_t)$.

In end-to-end reinforcement learning (RL) navigation algorithms, the action set A typically consists of direct control commands for the UAV, such as flight angles, velocities, or rotor speeds. Executing these control signals changes the state s, resulting in a reward r. However, in the proposed hierarchical and tightly coupled ETRLNav framework, the action set A comprises spatial coordinates. This action—specifically, the generation of a local target point—does not immediately cause the UAV to transition to the next state s_{t+1} or yield an immediate reward r_t. Within this hierarchical architecture, the action completes only the upper-level task; the UAV must then employ the motion planner to reach the selected local target to conclude a time step. Therefore, the transition to state s_{t+1}, along with receiving a new observation o_t and reward r_t, occurs only after the UAV has executed the planned trajectory to the target point. Consequently, the original POMDP is extended to accommodate this hierarchical structure.

In the ETRLNav framework, a successful navigation strategy consists of a sequence of local targets: $\tau_{\text{sus}} = \{p_0, \cdots, p_{\tau-1}\}$. Each local target corresponds to a configuration in the motion planner's space, representing the UAV's pose in a three-dimensional environment. At each policy step, the navigation algorithm invokes the motion planner to generate smooth, collision-free trajectories. The motion planner processes the local targets provided by the DRL strategy and outputs an action sequence that satisfies dynamic feasibility constraints.

a) Observation Space: The Local Goal Generation Policy (LoGP) processes raw sensor data to produce sub-targets for the motion planner (as illustrated in Figure 1). The UAV's internal state is characterized by two angles: ϑ, the angle between its forward direction and the x-axis, and φ, the angle between its forward direction and the horizontal plane. The initial relative position between the UAV and the goal is denoted by $\xi = [d_g, \varpi, \varpi_z]$. The inclusion of d_g provides a direct measure of the goal's proximity, which aids the navigation policy by distinguishing between varying distances to the target. This distance information ensures that the UAV can effectively prioritize goal-directed motion, especially in environments with dense obstacles. The UAV perceives its environment through an onboard RGB-D camera. The depth images are compressed using a Variational Autoencoder (VAE), and the compressed representations, denoted as $\mathcal{I}$, are utilized in training the Recurrent Soft Value Gradient (RSVG) algorithm. Therefore, the observation space is defined as $o = [\vartheta, \varphi, \mathcal{I}, \xi]$, and the output is the next desired local target.

b) Action Space: The LoGP treats the local target space as a continuous action space at the higher level. The policy network outputs a vector $a = [x, y, z, \phi, \theta, \psi]$, specifying a position and orientation in the UAV's body coordinate system, effectively indicating the desired end-effector pose. The parameter ψ conveys directional information about the predicted local target, which can be used to refine the motion trajectory. This refinement enables the lower-level motion planner to reduce unnecessary rotations, resulting in more natural and efficient movements.

c) Reward Design: To guide the agent in making rational decisions from limited sensory observations, we design a non-sparse reward function that balances two key objectives: encouraging goal-directed motion and ensuring obstacle avoidance. The reward comprises a distance reward and an obstacle penalty, each reflecting critical decision-making priorities.

The distance reward promotes forward progress and is formulated as:

$$r_{\text{dist}} = \omega_g \left(|p^{t+1} - g| - |p^t - g| \right) \tag{3}$$

where p^t and p^{t+1} denote the UAV's current and next positions, g is the goal position, and ω_g is a positive coefficient. A higher ω_g increases the incentive for aggressive movement toward the goal, accelerating navigation but potentially at the cost of safety. Thus, it should be selected based on the required mission urgency and environmental complexity.

The obstacle penalty is designed to prevent the UAV from selecting sub-goals too close to obstacles, even if they yield short-term progress:

$$r_{\text{obs}} = -e^{-\sigma d_{\min}} \tag{4}$$

where $d_{\min}$ is the minimum distance between the selected sub-goal and the nearest obstacle, and σ controls the sensitivity of the penalty. A larger σ results in stronger penalties even when obstacles are moderately close, encouraging conservative and safer behaviors. This term prevents the UAV from selecting risky sub-goals that might require overly sharp turns or lead to collisions under uncertain dynamics.

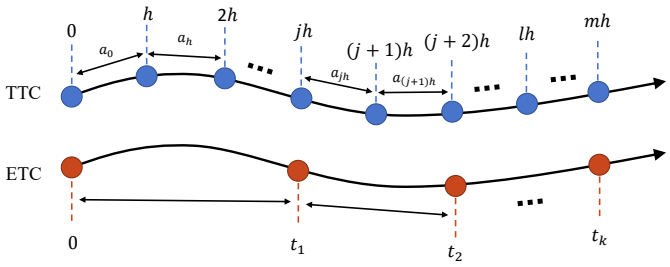


Fig. 3. Comparison of time-triggered and event-triggered control in trajectory planning.

The overall reward is:

$$r = r_{\text{dist}} + r_{\text{obs}} \tag{5}$$

The relative weighting of ω_g and σ enables flexible tuning between aggressive and cautious navigation strategies. In practice, we empirically tune these parameters to achieve high success rates while maintaining low jerk and acceleration, as detailed in the experimental analysis.

B. Event-Triggered RSVG Algorithm

For UAV navigation in unknown environments, the upper-level deep reinforcement learning algorithm must not only determine effective local target points but also avoid obstacles within its field of view. By incorporating an event-triggered mechanism that responds to environmental changes, navigation performance can be significantly enhanced. The event-triggered conditions are defined as the detection of obstacles that pose a potential threat to the UAV.

TTC operates by generating target points at fixed intervals, denoted by h, which in turn triggers decisions. In a hierarchical framework, once the upper-level strategy produces a target point, the lower-level motion planner executes the corresponding trajectory. In TTC, fixed intervals between upper-level decisions require equally spaced control points for the lower-level planner. In environments with sparse obstacles, frequent upper-level decisions may lead to unnecessary computational overhead and reduced efficiency. Conversely, in densely populated areas, longer intervals between decision points increase the risk of collisions before the next upper-level target is set. Thus, TTC demands constant parameter tuning based on environmental conditions to optimize the interaction between the upper and lower levels.

In contrast, ETC adjusts the frequency of target point generation dynamically in response to environmental changes. This reduces computational demands and increases operational efficiency. Specifically, an event is triggered when the minimum distance d_{obs} between the UAV and any obstacle within its field of view falls below a predefined threshold d_{ET}:

$$\gamma_k = \begin{cases} 1, & \text{if } d_{\text{obs}} \leq d_{\text{ET}} \\ 0, & \text{otherwise} \end{cases} \tag{6}$$

The threshold d_{ET} is a tunable hyperparameter controlling the sensitivity of the event trigger. A smaller d_{ET} allows fewer decisions and emphasizes energy efficiency, while a larger d_{ET} results in more frequent reactivity to nearby obstacles, improving safety in cluttered scenes. In our experiments, we empirically set $d_{\text{ET}} = 3.0\text{m}$, which balances trajectory smoothness, safety, and computational cost. This mechanism ensures that decisions are made only when necessary, enabling adaptive re-planning without constant control updates.

After an event is triggered, the local target generation strategy is trained using the Recurrent Soft Policy Gradient (RSPG) algorithm.

In partially observable environments, the optimal strategy depends on past trajectory information, denoted as h_t. Replacing the traditional feedforward network with a recurrent neural network enables the system to learn from past information. The relationship between observation history h_t and the underlying state s_t is expressed as $s_t \sim p(s_t|h_t)$. Taking the expectation over the state distribution $s_t \sim p(s_t|h_t)$, the soft Q-function under the maximum entropy framework is defined as:

$$\begin{aligned} Q^{\pi}\left(\mathbf{h}_t, \mathbf{a}_t\right) = \mathbb{E}_{s_t|h_t}\Big[r\left(\mathbf{s}_t, \mathbf{a}_t\right) \\ + \sum_{l=1}^{\infty} \gamma^l \mathbb{E}_{(\mathbf{s}_{t+l}, \mathbf{a}_{t+l}) \sim \rho_\pi}\left[r\left(\mathbf{s}_{t+l}, \mathbf{a}_{t+l}\right) + \mathcal{H}^{\pi}\left(\cdot \mid \mathbf{s}_{t+l}\right)\right]\Big] \end{aligned} \tag{7}$$

The policy update for continuous space POMDPs, following maximum entropy reinforcement learning, is:

$$\nabla_\theta J\left(\pi_\theta\right) = \mathbb{E}_\tau\left[\left(Q^{\pi}(\mathbf{h}, \mathbf{a}) - \log \pi(\mathbf{a}|\mathbf{h}) - 1\right) \nabla_\theta \log \pi(\mathbf{a}|\mathbf{h})\right] \tag{8}$$

The expectation across the entire trajectory τ is drawn from the distribution of the current policy π, and the historical trajectory h_t is represented as $(o_1, a_1, o_2, a_2, \ldots, a_{t-1}, o_t)$.

In practice, a learned approximation Q^{ω} is used to replace the true Q^{π}, and the soft policy gradient becomes:

$$\nabla_\theta J\left(\pi_\theta\right) = \mathbb{E}_\tau\left[\left(Q^{\omega}(\mathbf{h}, \mathbf{a}) - \log \pi(\mathbf{a}|\mathbf{h}) - 1\right) \nabla_\theta \log \pi(\mathbf{a}|\mathbf{h})\right] \tag{9}$$

During learning, the RSPG agent interacts with the environment using the current policy. The state information is stored in a replay buffer, and mini-batches are uniformly sampled from the buffer to update both the actor and critic network parameters.

The event-triggered mechanism poses challenges to the convergence speed of the RSPG algorithm. Since decisions and learning occur only when obstacles are detected, the agent may accumulate insufficient experience, thus slowing the learning process. Reinforcement learning depends on frequent environment interactions to maximize cumulative rewards through action selection. Reduced opportunities for interaction can negatively impact learning efficiency. Additionally, the parameters are updated based on historical trajectories rather than the full episode, exacerbating the slow convergence issue, particularly when using Stochastic Gradient Descent (SGD), as correlated histories may be sampled.

To address this, the Fast RSPG (FRSPG) algorithm is proposed, which improves experience utilization by combining event-triggered decisions with historical data updates. As de-

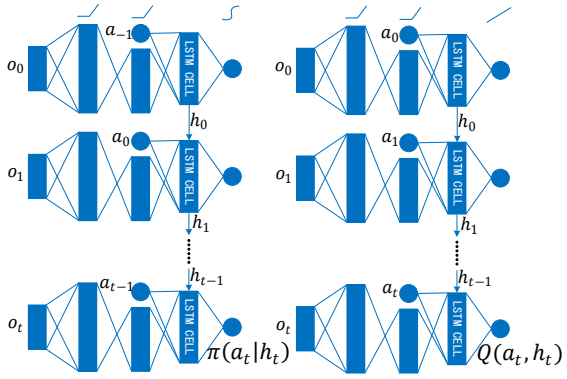


Fig. 4. Schematic representation of the actor-critic network with localized target points for decision making

rived in [3], the policy updated based on historical trajectories is expressed as:

$$\eta(\pi) = \mathbb{E}_{h_0}\left[V_\pi(h_0)\right], \tag{10}$$

The original RSPG policy gradient then becomes:

$$\begin{aligned}\nabla_\theta J\left(\pi_\theta\right) =& \mathbb{E}_{h\sim d_\pi(h)}\mathbb{E}_{a\sim\pi(a|h)}\left[(Q^\pi(\mathbf{h},\mathbf{a})\right.\\ &\left.-\log\pi(\mathbf{a}\mid\mathbf{h})-1)\nabla_\theta\log\pi(\mathbf{a}\mid\mathbf{h})\right]\end{aligned} \tag{11}$$

The stochastic gradient updates are no longer dependent on full trajectories, allowing for rapid online updates based on historical data.

C. Motion Planner

The lower-level motion planner employs a lightweight gradient-based EGO-planner [24], which optimizes the trajectory by storing obstacle information only when the current path collides with newly detected obstacles, minimizing computation while ensuring trajectory optimization. The planner generates and optimizes trajectories by initially parameterizing them as uniform B-spline curves, represented by N control points $\{\mathbf{Q}_1, \mathbf{Q}_2, \cdots, \mathbf{Q}_N\}$ and a knot vector $\{t_1, t_2, \cdots, t_M\}$, where $\mathbf{Q}_i \in \mathbb{R}^3$ and $t_m \in \mathbb{R}$. Following the method outlined in [**?**], the control points are reduced to a differentially flat output space. The optimization problem is formulated as:

$$\min_{\mathbf{Q}} J = \lambda_s J_s + \lambda_c J_c + \lambda_d J_d \tag{12}$$

where J_s represents the smoothness penalty, J_c the collision penalty, J_d the dynamic feasibility term, and λ_s, λ_c, λ_d are the corresponding weight factors. If the generated trajectory violates dynamic constraints, the trajectory time is reallocated, and the optimization process is repeated.

D. Computational Complexity Analysis

In real-world UAV applications, real-time performance under limited onboard resources is crucial. ETRLNav addresses this by employing event-triggered decision-making and a lightweight planner, significantly reducing redundant computation [25], [26].

Theoretical analysis shows that standard end-to-end RL incurs a complexity of $O(T_E(H^2 + HD))$, where T_E is the number of time steps, H the hidden size, and D the input dimension. In contrast, ETRLNav triggers decisions only at key moments with frequency $f_T \ll 1$, resulting in reduced upper-level complexity: $\mathcal{C}_{\text{ETDRL}} = O(f_T T_E(H^2 + HD))$.

The lower-level B-spline planner adds a cost of $\mathcal{C}_{\text{Planner}} = O(mkn)$, where m, k, and n denote planning calls, optimization steps, and control points, respectively. Thus, the overall complexity $\mathcal{C}_{\text{ETRLNav}} = O(f_T T_E(H^2 + HD) + mkn)$ remains suitable for real-time UAV deployment.

To further demonstrate the feasibility of ETRLNav for real-time UAV navigation, we compare its computational complexity with other representative DRL-based methods. For instance, RSPG [9] requires executing the policy network and a subgoal sampler at every step, resulting in a complexity of $O(T_E(H^2 + HD + S))$, where S denotes the cost of subgoal sampling. LB-WayPtNav [19] performs periodic waypoint generation and motion planning with fixed frequency f_{LB}, yielding a total complexity of $O(f_{LB}T_E(H^2 + HD) + m_{LB}k_{LB}n_{LB})$. ReLMoGen [20], which includes an end-to-end subgoal generator and motion generator, incurs even higher complexity due to additional perception processing. ETHP [21] uses dual deep networks (for planning and tracking), both triggered at high frequency, leading to cumulative complexity $O(T_E(2H^2 + 2HD))$. In contrast, ETRLNav dynamically reduces upper-level network calls and decouples motion execution, achieving superior efficiency. These comparisons highlight that ETRLNav achieves real-time feasibility while preserving decision quality, making it suitable for onboard deployment in constrained UAV platforms.

V. Experimental Results

A. Simulation Setup

The simulation environment, illustrated in Figure 5, is based on the ROS environment and generated using MockaMap [27]. Four types of environments were created: randomly generated square pillars, cylindrical pillars, tori, and irregularly shaped 3D obstacles resembling fog. These simulate real-world obstacles like urban buildings and forest trees. The environment size is 40x40x12 meters, with obstacle areas spanning 35×35×12 meters, ensuring that the UAV starts and ends in safe, unobstructed spaces. Obstacle density is defined by the percentage of space volume they occupy, where increasing obstacle volume adds complexity to UAV navigation, often creating local traps like those in Figure 1. We evaluate the algorithms under various obstacle densities, with a focus on high-density scenarios mimicking real-world conditions. All simulations are conducted on Ubuntu 16.04 with 32GB RAM, an AMD R5-5600G CPU, and a GeForce GTX 1080 GPU. UAV training spans 10,000 time steps (approximately 10 hours). The UAV's initial position, environmental obstacles, and target locations are randomized during training to prevent overfitting. The trained policies are tested in scenarios with different obstacle densities and configurations.

Model hyperparameters are fine-tuned through repeated experiments to select the best-performing configuration. The

UAV's maximum speed is set to 1.5 m/s, with a flight altitude between 0.0m and 30.0m. Reward parameters are configured as $\sigma = 2.0$, $\alpha = 8.0$, $\beta = 25.0$, $r_{\text{free}} = 0.1$, $r_{\text{step}} = -0.6$. The maximum length of historical estimation is set to 100 steps. The learning rate of the Adam Optimizer is 0.01, with a target network update rate of $\tau = 0.01$. The replay buffer size is 10^5, sampled every 100 time steps. The discount factor is $\gamma = 0.99$. Table VI summarizes the key parameters involved in the proposed framework.

The DRL policy is trained in a simulation environment, where the UAV learns optimal navigation strategies through reward signals related to goal proximity, obstacle avoidance, and trajectory smoothness. Once trained, the policy is transferred to the UAV by exporting the neural network weights. During real-time operation, the UAV uses onboard resources to execute the policy and make navigation decisions. The policy is lightweight to minimize computational overhead. A communication protocol ensures that the UAV's policy can be updated remotely when needed, enabling continuous learning with minimal latency.

B. Compared Methods

To verify the effectiveness of the proposed ETRLNav framework, we compare it with several representative baselines:

(1) End-to-End Methods: We include RSPG and DSPG, which are widely used DRL methods for UAV navigation. These methods directly map raw sensory input to control commands and serve as benchmarks for end-to-end policy learning. The implementation follows the works of Lillicrap et al. (DDPG) [7] and the soft actor-critic-based adaptation in RSPG [3].

(2) Classical Planning Methods: We implement a geometry-based approach combining occupancy grid mapping with A* global planning and a local ego-planner [24], which is representative of traditional reactive planning frameworks used in UAV systems.

(3) Recent Hierarchical or Event-Triggered Methods: To benchmark against recent advancements, we additionally compare ETRLNav with the following three state-of-the-art methods:

a) LB-WayPtNav [19]: A learning-based high-level waypoint planner combined with a model-based controller, designed for ground robots. We adapt it to the UAV domain for fair evaluation.

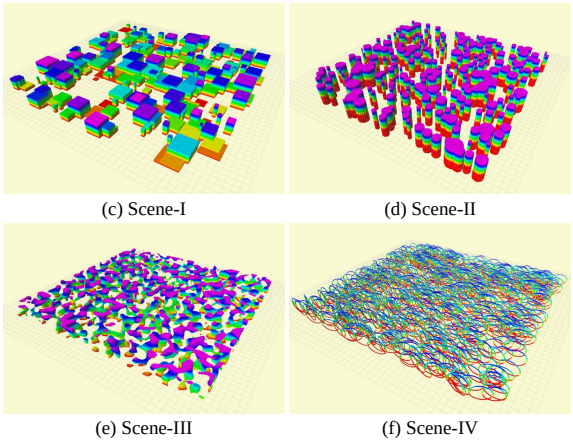
b) ReLMoGen [20]: A reinforcement learning framework that integrates local motion generation, supporting better decision-making in high-dimensional observation space.

c) ETHP [21]: A recent event-triggered hierarchical approach designed for UAVs, which uses deep neural networks for both planning and control layers.

These baselines were either re-implemented or adapted to the same ROS-based simulation environment for consistent evaluation. Detailed configurations and hyperparameter settings are included in the supplementary materials.

The algorithms are evaluated using the following metrics:

a) Average reward: The average reward from 200 navigation tasks in four environments is used to evaluate overall performance.



(c) Scene-I (d) Scene-II

(e) Scene-III (f) Scene-IV

Fig. 5. Schematic diagram of simulation environment

b) Success rate: The percentage of UAVs that successfully reach the target without collisions.

c) Trapping rate: The percentage of agents failing to reach the target within the allotted time.

d) Collision rate: The percentage of agents that fail due to collisions.

e) Average acceleration and jerk: Metrics assessing smoothness and energy consumption.

f) Average curvature: Measures the change in flight direction per unit distance, reflecting the UAV's ability to maintain a smooth turning profile.

g) Heading rate: Quantifies the rate of directional change over time, indicating the angular agility of the UAV during navigation.

h) Trigger Frequency: To measure ETRLNav's efficiency, we introduce the trigger frequency $\mathcal{P} = \frac{\sum_{t=0}^{N} \mathbb{I}}{N}$, where $\mathbb{I}$ represents the number of event triggers, and N is the total number of steps.

C. Performance of the ETRLNav Navigation Framework

a) Average Reward: Figure 6 displays the average training rewards for ETRLNav, RSPG, and DSPG. The rewards are averaged over 250 training episodes. The results demonstrate that ETRLNav achieves better sampling efficiency and overall reward compared to the other two algorithms. ETRLNav converged within 9500 episodes, which is 3500 episodes faster than RSPG. This speed advantage is attributed to ETRLNav's ability to focus on identifying high-reward local targets without needing to learn basic flight mechanics. Additionally, ETRLNav demonstrates greater stability than DSPG in complex environments with partially observable obstacles, as it leverages memory through recursive networks.

b) Performance of ETRLNav in Different Scenarios: The impact of different environments on UAV performance was assessed using 100 successful flight tests across four scenarios with 50% obstacle density and dimensions of 50m $\times$ 50m. Table I shows that ETRLNav's flight time remained consistent (40 seconds) in cylindrical, foggy, and toroidal environments, with a flight speed of approximately 1.4 m/s.

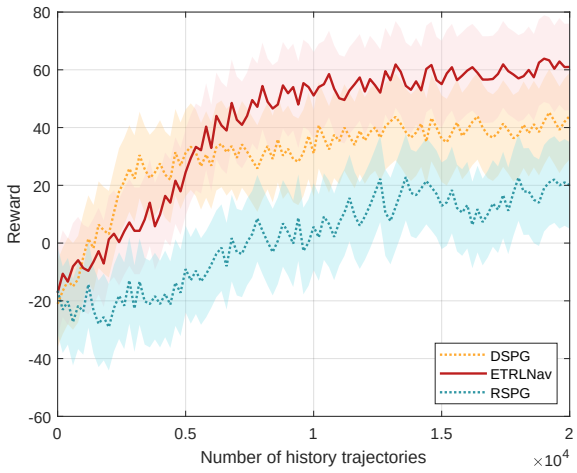


Fig. 6. Schematic representation of the average rewards of the three algorithms.

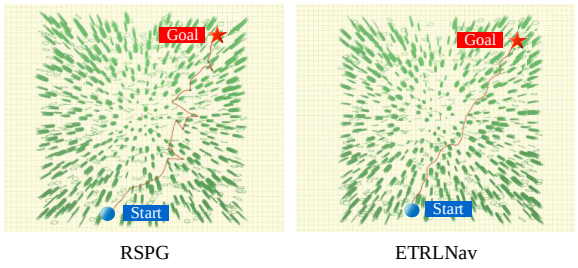


Fig. 7. Comparison of navigation trajectories generated by the ETRLNav algorithm with the end-to-end RSPG algorithm

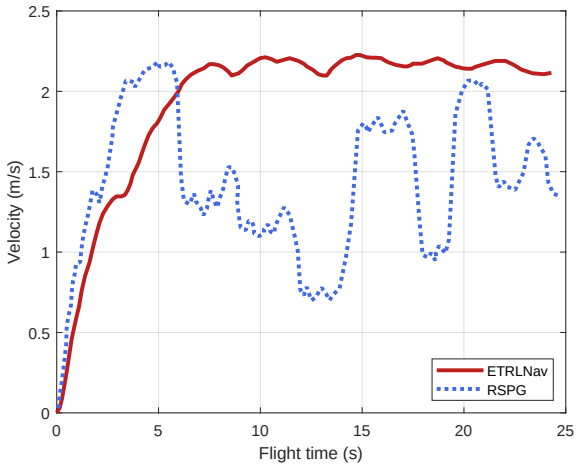


Fig. 8. Velocity change profile of ETRLNav algorithm with end-to-end RSPG algorithm navigation.

In scenarios with cubic obstacles, flight time increased by about 10 seconds compared to other environments. This was due to cubic obstacles hindering the drone's visual perception with right-angled edges, creating additional challenges. As a result, ETRLNav took longer in these scenarios. Similarly, flight path efficiency dropped by around 7% in cubic environments due to the more complex navigation required.

To ensure a fair and comprehensive comparison, we evaluate the proposed ETRLNav framework alongside three recent hierarchical or event-triggered navigation methods from the past five years. As shown in Table I, ETRLNav achieves the highest success rate (99.25%) and lowest collision rate (0.00%), while maintaining low acceleration and jerk. These results highlight its superior safety, efficiency, and smoothness in complex environments.

D. Compared with reinforcement learning methods

As shown in Table II, ETRLNav achieves a 15% higher success rate than the end-to-end RSVG method. Additionally, the time to reach the target is reduced by 24%, and average acceleration is lowered by 29%, indicating lower energy consumption. Figure 7 illustrates the navigation trajectories generated by both methods. The ETRLNav framework delegates trajectory generation and optimization to the motion planner, allowing the upper-layer RL algorithm to focus solely on selecting local targets. This enables the UAV to navigate narrow passages and sharp turns more effectively than the end-to-end method.

Velocity control curves (Figure 8) show that the RSVG method exhibits abrupt speed changes during testing, while ETRLNav produces smoother trajectories with smaller acceleration variations. The low acceleration of ETRLNav conserves energy, enhancing endurance.

In addition to acceleration and jerk, we also evaluate trajectory curvature and heading rate to provide a more comprehensive assessment of trajectory smoothness. As shown in Table I, ETRLNav achieves a significantly lower average curvature (0.042) and heading rate (0.072) compared to the end-to-end RSPG method, which records values of 0.118 and 0.154 respectively. This indicates that the UAV under ETRLNav follows smoother and more natural turns, avoiding sharp deviations and excessive directional oscillations. These improvements reflect the benefits of combining event-triggered waypoint selection with optimized motion planning, which enables ETRLNav to produce dynamically feasible and comfortable trajectories for real-world navigation.

E. Comparison with mapping and planning methods

Non-learning-based methods rely on occupancy maps derived from RGB-D images, which are processed by a gradient-based motion planner for control generation. Table II shows that ETRLNav outperforms these methods in success and entrapment rates. The heuristic A* algorithm used in non-learning approaches often fails in complex environments, leading to a higher entrapment rate (5.75%) compared to ETRLNav (1.25%). In terms of trajectory smoothness, both methods perform similarly, generating paths with gradual speed changes.

F. Comparison with Variant Methods

The ETRLNav algorithm was compared with its variant, RLNav-nonET, which lacks event-triggering, through 100 flight tests across four scenarios, all starting from the same initial point. The objective was to statistically evaluate the

TABLE I
COMPARISON WITH STATE-OF-THE-ART NAVIGATION METHODS

Method	Success Rate (%)	Collision Rate (%)	Trapped Rate (%)	Flight Time (s)	Accel. (m/s^2)	Jerk (m/s^3)	Curvature (rad/m)	Heading Rate (rad/s)
ETRLNav (Ours)	**99.25**	**0.00**	0.75	17.83±2.97	0.12±0.01	3.56±0.14	0.031±0.006	0.42±0.08
LB-WayPtNav [19]	91.75	4.50	3.75	19.42±3.15	0.16±0.01	3.92±0.15	0.037±0.007	0.48±0.09
ReLMoGen [20]	89.50	6.25	4.25	21.67±3.75	0.19±0.02	4.23±0.18	0.051±0.012	0.53±0.10
ET-DRLNav [21]	96.25	2.50	1.75	18.96±2.84	0.17±0.03	3.88±0.14	0.041±0.004	0.46±0.08
RLNav-nonET	94.50	4.25	1.25	18.54±3.62	0.13±0.01	3.62±0.13	0.034±0.005	0.45±0.07
RSPG (E2E)	87.25	8.25	4.50	27.34±4.46	0.41±0.03	7.64±0.51	0.086±0.011	0.91±0.12
EGO-Planner	84.50	9.75	5.75	20.79±2.85	0.13±0.01	3.52±0.16	0.032±0.006	0.41±0.06

TABLE II
ETRLNAV AND RLNAV-NONET ALGORITHMS FOR DECISION-MAKING TOUCH FREQUENCIES IN FOUR SCENARIOS

Decision frequency	SCENE-I	II	III	IV
ETRLNav	29	18	21	13
RLNav-NonET	45	31	37	25

average decision-making frequency of the upper-level reinforcement learning algorithms. The experimental results, summarized in Table II, show that ETRLNav required fewer target point decisions in all four scenarios compared to RLNav-nonET. This indicates that ETRLNav is more efficient in terms of computational resource usage in obstacle-triggered situations. Additionally, Table II shows a significantly lower collision rate for ETRLNav, highlighting its enhanced safety.

G. Generalization Ability

Obstacle density and map size significantly influence UAV performance. Figure 9(a) and (b) show that as obstacle density increases, the success rates of all three algorithms decline. However, ETRLNav maintains a success rate of approximately 90%, while the RSVG method drops to 67% at 60% obstacle density. Entrapment rates also increase with obstacle density, with RSVG showing the highest rate at 10%. Additionally, trigger frequency for ETRLNav rises in denser environments, as more local target points need re-specification.

Expanding the environment size further reduces success rates, as shown in Figures 9(c) and (d). Despite this, ETRLNav maintains a success rate of around 82%, outperforming RSVG, which struggles with long-distance target navigation due to its end-to-end nature.

H. Robustness to Sensor Noise and Decision Latency

In real-world UAV deployments, sensor inputs may be corrupted by noise, and decision-making can be delayed due to onboard processing or communication overhead. To evaluate the practical robustness of the proposed ETRLNav framework, we conducted additional experiments under controlled sensor noise and action latency conditions.

We injected zero-mean Gaussian noise into the UAV's depth observations with standard deviations $\sigma \in 0.01, 0.05, 0.10$. As summarized in Table III, ETRLNav retained a high success rate of 86.00% even under strong noise ($\sigma = 0.10$), with only modest increases in average acceleration (0.18 m/s^2) and

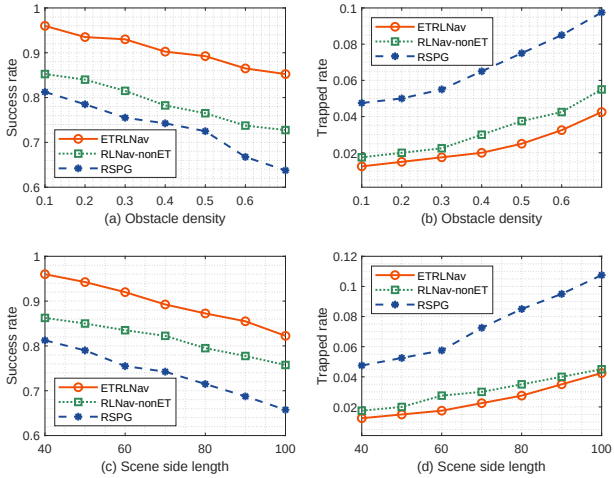


Fig. 9. Performance of three algorithms for different obstacle environment sizes and densities.

jerk (4.30 m/s^3) compared to the baseline. This confirms the robustness of the hierarchical architecture, where the motion planner compensates for noisy decisions from the high-level module.

To simulate system-level delays, we introduced execution lags of $\delta \in 3, 5$ steps between decision and actuation. While performance slightly declined, the success rate remained above 88.00%, with average acceleration and jerk increasing only marginally to 0.17 m/s² and 4.20 m/s³, respectively. These results indicate that ETRLNav is resilient to latency, benefiting from its event-triggered mechanism that reduces dependence on frequent updates.

I. Impact of Sensor Range and Trigger Distance

The performance of event-triggered navigation inherently depends on the UAV's perception capabilities and the sensitivity of the triggering mechanism. To systematically analyze these factors, we conducted additional experiments by varying both the sensor perception range and the event-trigger distance, aiming to evaluate their influence on navigation efficiency and safety.

We simulated four different field-of-view ranges: 6 m, 8 m, 10 m, and 12 m. As shown in Table IV, a larger perception range significantly improves the UAV's situational awareness, enabling it to identify obstacles earlier and plan smoother, safer trajectories. For example, the success rate increased

TABLE III
ROBUSTNESS OF ETRLNAV UNDER SENSOR NOISE AND DECISION LATENCY

Condition	Level	Success Rate (%)	Avg Acc. (m/s^2)	Avg Jerk (m/s^3)
Sensor Noise (Gaussian)				
$\sigma = 0.00$	baseline	99.25	0.12	3.56
$\sigma = 0.01$	mild	97.00	0.13	3.65
$\sigma = 0.05$	moderate	89.50	0.15	3.92
$\sigma = 0.10$	strong	86.00	0.18	4.30
Decision Delay (steps)				
$\delta = 0$	no delay	99.25	0.12	3.56
$\delta = 3$	medium	94.50	0.14	3.75
$\delta = 5$	high	88.00	0.17	4.20

TABLE IV
IMPACT OF PERCEPTION RANGE AND TRIGGER DISTANCE ON NAVIGATION PERFORMANCE

Condition	Success Rate (%)	Collision Rate (%)	Avg Acc. (m/s^2)	Avg Jerk (m/s^3)
Perception Range (m)				
6	85.00	8.00	0.19	4.41
8	89.75	5.25	0.16	3.91
10	92.00	3.75	0.14	3.66
12	95.25	2.25	0.13	3.48
Trigger Distance (m)				
3	87.00	8.00	0.18	4.35
4	91.00	5.50	0.16	3.92
5	94.50	3.25	0.14	3.66
6	92.00	4.00	0.15	3.78
7	90.25	5.00	0.17	4.02

from 85.00% (at 6 m) to 95.25% (at 12 m), while average acceleration dropped from 0.19 to 0.13. This demonstrates that richer sensory input improves planning quality and reduces reactive maneuvers. Conversely, smaller FOVs restrict perception, increasing the risk of late obstacle detection and suboptimal sub-goal selection.

The event-trigger distance controls how soon a decision is made in response to perceived obstacles. We tested three values: 3 m, 5 m, and 7 m. A very short trigger distance (3 m) delays reactions until the UAV is near obstacles, resulting in more frequent collisions (collision rate: 8.00%) and abrupt trajectory corrections. Conversely, an overly long trigger distance (7 m) leads to premature and sometimes unnecessary replanning, which marginally increases jerk and computational effort. A medium setting of 5 m achieves the best balance, maintaining a 94.50% success rate with low acceleration (0.14) and jerk (3.66), suggesting that moderate sensitivity yields both responsive and stable behavior.

These results confirm that both perception range and trigger threshold must be carefully tuned to ensure reliable, efficient navigation in complex environments.

J. Inter-Module Communication Efficiency

In hierarchical architectures, the latency between high-level decision-making and low-level execution may pose potential bottlenecks, particularly under frequent communication. To assess the responsiveness of our framework, we profiled the interaction efficiency between the DRL sub-goal generator and the motion planner.

TABLE V
INTER-MODULE LATENCY PROFILING AND COMMUNICATION EFFICIENCY

Metric	Value
Sub-goal generation latency	14.7 ms
Trajectory optimization latency	26.3 ms
Control cycle threshold	50 ms
Communication reduction (vs. TTC)	35.6%

Experimental results, averaged over 500 navigation episodes in Scene-II (obstacle density 40%), show that the average latency for sub-goal generation is 14.7 ms, and trajectory optimization takes 26.3 ms, both well within the standard control cycle of 50 ms. Additionally, thanks to the event-triggered mechanism, the frequency of high-level decisions is significantly reduced—by 35.6% compared to the time-triggered baseline—without degrading navigation performance, as summarized in Table V.

This reduction not only alleviates bandwidth and synchronization pressure but also ensures real-time responsiveness of the decision-feedback loop. The results confirm that the proposed event-triggered feedback coordination enables stable and efficient execution for onboard deployment.

TABLE VI
KEY PARAMETERS IN THE REINFORCEMENT LEARNING FRAMEWORK

Parameter	Description
Observation vector: $\xi = [d_g, \varpi, \varpi_z]$	distance to goal, azimuth, and elevation angles
Action vector: $a = [x, y, z, \phi, \theta, \psi]$	UAV's pose
The UAV's maximum speed	1.5 m/s
Flight altitude range	0.0 m to 30.0 m
Maximum historical estimation length	100 steps
Target network update rate τ	0.01
Replay buffer size	10^5
Discount factor γ	0.99

VI. CONCLUSION

This work introduces ETRLNav, an event-triggered hierarchical navigation framework for UAVs that combines deep reinforcement learning for sub-goal selection with classical motion planning. By triggering decisions only when environmental changes are detected, ETRLNav reduces redundant computation and enhances real-time responsiveness. Compared with end-to-end and fixed-interval methods, it demonstrates superior success rates, trajectory smoothness, and generalization in complex environments.

The event-triggered mechanism provides a scalable and resource-efficient solution for real-world UAV applications. Its modular design and lightweight planning enable onboard

deployment under limited compute and energy budgets, making it suitable for industrial applications such as inspection, logistics, and disaster response. Furthermore, the proposed framework achieves efficient performance even in high-density or large-scale scenarios, as validated by both theoretical complexity analysis and empirical results.

Although this study focuses on single-UAV navigation in static layouts, the framework naturally extends to multi-UAV settings. Event-triggered decision-making can be decentralized, allowing each UAV to act autonomously while reducing communication overhead. Future work will explore dynamic obstacle modeling, predictive planning, and multi-agent reinforcement learning, incorporating intention-aware coordination and sub-goal consensus strategies to support collaborative navigation and conflict avoidance in dynamic, shared spaces.

REFERENCES

[1] G. Raja, A. Manoharan, and H. Siljak, "Ugen: Uav and gan-aided ensemble network for post-disaster survivor detection through oran," *IEEE Transactions on Vehicular Technology*, vol. 73, no. 7, pp. 9296–9305, 2024.

[2] W. Lee, B. Shahzaad, B. Alkouz, and A. Bouguettaya, "Reactive composition of uav delivery services in urban environments," *IEEE Transactions on Intelligent Transportation Systems*, 2024.

[3] Y. Xue and W. Chen, "Multi-agent deep reinforcement learning for uavs navigation in unknown complex environment," *IEEE Transactions on Intelligent Vehicles*, vol. 9, no. 1, pp. 2290–2303, 2023.

[4] H. Nawaz, H. M. Ali, and A. A. Laghari, "Uav communication networks issues: a review," *Archives of Computational Methods in Engineering*, vol. 28, pp. 1349–1369, 2021.

[5] Y. Xue and W. Chen, "A uav navigation approach based on deep reinforcement learning in large cluttered 3d environments," *IEEE Transactions on Vehicular Technology*, vol. 72, no. 3, pp. 3001–3014, 2022.

[6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[7] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[8] N. Heess, J. J. Hunt, T. P. Lillicrap, and D. Silver, "Memory-based control with recurrent neural networks," *arXiv preprint arXiv:1512.04455*, 2015.

[9] H. X. Pham, H. M. La, D. Feil-Seifer, and L. Van Nguyen, "Reinforcement learning for autonomous uav navigation using function approximation," in *2018 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2018, pp. 1–6.

[10] J. De Heuvel, W. Shi, X. Zeng, and M. Bennewitz, "Subgoal-driven navigation in dynamic environments using attention-based deep reinforcement learning," in *2023 21st International Conference on Advanced Robotics (ICAR)*. IEEE, 2023, pp. 79–85.

[11] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*. MIT press, 2022.

[12] T. Siméon, J.-P. Laumond, and C. Nissoux, "Visibility-based probabilistic roadmaps for motion planning," *Advanced Robotics*, vol. 14, no. 6, pp. 477–493, 2000.

[13] S. M. LaValle, J. J. Kuffner, B. Donald *et al.*, "Rapidly-exploring random trees: Progress and prospects," *Algorithmic and computational robotics: new directions*, vol. 5, pp. 293–308, 2001.

[14] J. Chen, K. Su, and S. Shen, "Real-time safe trajectory generation for quadrotor flight in cluttered environments," in *2015 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. IEEE, 2015, pp. 1678–1685.

[15] F. Gao and S. Shen, "Online quadrotor trajectory generation and autonomous navigation on point clouds," in *2016 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR)*. IEEE, 2016, pp. 139–146.

[16] H. Oleynikova, Z. Taylor, R. Siegwart, and J. Nieto, "Safe local exploration for replanning in cluttered unknown environments for microaerial vehicles," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1474–1481, 2018.

[17] K. Rana, B. Talbot, V. Dasagi, M. Milford, and N. Sünderhauf, "Residual reactive navigation: Combining classical and learned navigation strategies for deployment in unknown environments," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 11 493–11 499.

[18] A. Faust, K. Oslund, O. Ramirez, A. Francis, L. Tapia, M. Fiser, and J. Davidson, "Prm-rl: Long-range robotic navigation tasks by combining reinforcement learning and sampling-based planning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 5113–5120.

[19] S. Bansal, V. Tolani, S. Gupta, J. Malik, and C. Tomlin, "Combining optimal control and learning for visual navigation in novel environments," in *Conference on Robot Learning*. PMLR, 2020, pp. 420–429.

[20] F. Xia, C. Li, R. Martín-Martín, O. Litany, A. Toshev, and S. Savarese, "Relmogen: Integrating motion generation in reinforcement learning for mobile manipulation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 4583–4590.

[21] C. Chen, B. Song, Q. Fu, D. Xue, and L. He, "Event-triggered hierarchical planner for autonomous navigation in unknown environment," *Drones*, vol. 7, no. 12, 2023. [Online]. Available: https://www.mdpi.com/2504-446X/7/12/690

[22] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and Z. Gao, "Uav trajectory planning for aoi-minimal data collection in uav-aided iot networks by transformer," *IEEE Transactions on Wireless Communications*, vol. 22, no. 2, pp. 1343–1358, 2022.

[23] Y. Kantaros, S. Kalluraya, Q. Jin, and G. J. Pappas, "Perception-based temporal logic planning in uncertain semantic maps," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 2536–2556, 2022.

[24] X. Zhou, Z. Wang, H. Ye, C. Xu, and F. Gao, "Ego-planner: An ESDF-free gradient-based local planner for quadrotors," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 478–485, 2020.

[25] C. Wang, R. Zhao, Y. Liu, Y. Liang *et al.*, "A lightweight progressive joint transfer ensemble network inspired by the markov process for imbalanced mechanical fault diagnosis," *Mechanical Systems and Signal Processing*, vol. 224, p. 111994, 2025.

[26] C. Wang, W. Zhang, G. Wang, and Y. Liu, "An energy-efficient mechanical fault diagnosis method based on neural dynamics-inspired metric spikingformer for insufficient samples in industrial internet of things," *IEEE Internet of Things Journal*, 2024.

[27] William.Wu, "mockamap," https://github.com/HKUST-Aerial-Robotics/mockamap .

Weisheng Chen received the B.Sc. degree from Qufu Normal University, Qufu, China, in 2000 and the M.Sc. and Ph.D. degrees from Xidian University, Xi'an, China, in 2004 and 2007, respectively. From 2013 to 2014, he was a Visiting Scholar with the Department of Electrical Engineering, University of California, Riverside, USA and is currently a Professor with the School of Information Mechanics and Sensing Engineering, Xidian University, Xi'an, China. He has authored or coauthored more than 150 journal and conference publications. His research interests include artificial intelligence, autonomous systems, nonlinear dynamics and control.

Yuntao Xue received the B.Sc. degree from Taiyuan University of Technology, Taiyuan, China, in 2017 and Ph.D. degrees from Xidian University, Xi'an, China, in 2024. He is currently working in the College of Electronic Information Engineering, Taiyuan University of Technology, Taiyuan, China.

His research interests include UAV navigation and deep reinforcement learning.